

Security Audit

Report for AlphaX - Protocol - Contracts

Date: Jan 26, 2025 **Version:** 1.0

Contact: contact@blocksec.com

Contents

Chapter 1 Introduction	1
1.1 About Target Contracts	1
1.2 Disclaimer	1
1.3 Procedure of Auditing	2
1.3.1 Software Security	2
1.3.2 DeFi Security	2
1.3.3 NFT Security	2
1.3.4 Additional Recommendation	3
1.4 Security Model	3
Chapter 2 Findings	4
2.1 DeFi Security	4
2.1.1 Potential risk of asset loss due to front-running	4
2.1.2 Bypassable requestId check in the <code>spotSwap()</code> function	6
2.1.3 Lack of whitelist check in the <code>depositETH()</code> function	7
2.2 Additional Recommendation	8
2.2.1 Remove unused functions and events	8
2.2.2 Implement balance validation logic in the <code>withdrawERC20()</code> function	8
2.2.3 Add non-zero value checks for the amount parameter in some functions	8
2.2.4 Avoid using the <code>transfer()</code> function for native token transfers	9
2.2.5 Add a check to validate the <code>exchangeData</code> parameter	10
2.2.6 Add <code>_disableInitializers()</code> in the <code>constructor()</code> functions	12
2.3 Note	12
2.3.1 Previously signed withdrawal transactions might become invalid after modifying important parameters	12
2.3.2 The <code>dailyWithdrawLimit</code> should be set immediately after configuring the <code>tokenWithdrawLimit</code>	12
2.3.3 Potential centralization risk	13

Report Manifest

Item	Description
Client	AlphaX-Protocol
Target	AlphaX-Protocol-Contracts

Version History

Version	Date	Description
1.0	Jan 26, 2025	First release

Signature

About BlockSec BlockSec focuses on the security of the blockchain ecosystem and collaborates with leading DeFi projects to secure their products. BlockSec is founded by top-notch security researchers and experienced experts from both academia and industry. They have published multiple blockchain security papers in prestigious conferences, reported several zero-day attacks of DeFi applications, and successfully protected digital assets that are worth more than 14 million dollars by blocking multiple attacks. They can be reached at [Email](#), [Twitter](#) and [Medium](#).

Chapter 1 Introduction

1.1 About Target Contracts

Information	Description
Type	Smart Contract
Language	Solidity
Approach	Semi-automatic and manual verification

This audit focuses on the code repositories of the AlphaX - Protocol - Contracts ¹ of AlphaX - Protocol.

Please note that this audit covers only the following contracts:

- contracts/DEXVaultRouter.sol
- contracts/DEXSpotVault.sol
- contracts/DEXVaultV2.sol

The auditing process is iterative. Specifically, we would audit the commits that fix the discovered issues. If there are new issues, we will continue this process. The commit SHA values during the audit are shown in the following table. Our audit report is responsible for the code in the initial version (Version 1), as well as new code (in the following versions) to fix issues in the audit report.

Project	Version	Commit Hash
AlphaX - Protocol - Contracts	Version 1	6ea8473c0d22c5a6d04efbca3915c79fb310c4b6

1.2 Disclaimer

This audit report does not constitute investment advice or a personal recommendation. It does not consider, and should not be interpreted as considering or having any bearing on, the potential economics of a token, token sale or any other product, service or other asset. Any entity should not rely on this report in any way, including for the purpose of making any decisions to buy or sell any token, product, service or other asset.

This audit report is not an endorsement of any particular project or team, and the report does not guarantee the security of any particular project. This audit does not give any warranties on discovering all security issues of the smart contracts, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues. As one audit cannot be considered comprehensive, we always recommend proceeding with independent audits and a public bug bounty program to ensure the security of smart contracts.

The scope of this audit is limited to the code mentioned in Section 1.1. Unless explicitly specified, the security of the language itself (e.g., the solidity language), the underlying compiling toolchain and the computing infrastructure are out of the scope. ®

¹<https://github.com/AlphaX-Protocol/AlphaX-Protocol-Contract>

1.3 Procedure of Auditing

We perform the audit according to the following procedure.

- **Vulnerability Detection** We first scan smart contracts with automatic code analyzers, and then manually verify (reject or confirm) the issues reported by them.
- **Semantic Analysis** We study the business logic of smart contracts and conduct further investigation on the possible vulnerabilities using an automatic fuzzing tool (developed by our research team). We also manually analyze possible attack scenarios with independent auditors to cross-check the result.
- **Recommendation** We provide some useful advice to developers from the perspective of good programming practice, including gas optimization, code style, and etc.

We show the main concrete checkpoints in the following.

1.3.1 Software Security

- * Reentrancy
- * DoS
- * Access control
- * Data handling and data flow
- * Exception handling
- * Untrusted external call and control flow
- * Initialization consistency
- * Events operation
- * Error-prone randomness
- * Improper use of the proxy system

1.3.2 DeFi Security

- * Semantic consistency
- * Functionality consistency
- * Permission management
- * Business logic
- * Token operation
- * Emergency mechanism
- * Oracle security
- * Whitelist and blacklist
- * Economic impact
- * Batch transfer

1.3.3 NFT Security

- * Duplicated item
- * Verification of the token receiver
- * Off-chain metadata security

1.3.4 Additional Recommendation

- * Gas optimization
- * Code quality and style



Note The previous checkpoints are the main ones. We may use more checkpoints during the auditing process according to the functionality of the project.

1.4 Security Model

To evaluate the risk, we follow the standards or suggestions that are widely adopted by both industry and academy, including OWASP Risk Rating Methodology ² and Common Weakness Enumeration ³. The overall *severity* of the risk is determined by *likelihood* and *impact*. Specifically, likelihood is used to estimate how likely a particular vulnerability can be uncovered and exploited by an attacker, while impact is used to measure the consequences of a successful exploit.

In this report, both likelihood and impact are categorized into two ratings, i.e., *high* and *low* respectively, and their combinations are shown in Table 1.1.

Table 1.1: Vulnerability Severity Classification

Impact	High	High	Medium
	Low	Medium	Low
		High	Low
		Likelihood	

Accordingly, the severity measured in this report are classified into three categories: **High**, **Medium**, **Low**. For the sake of completeness, **Undetermined** is also used to cover circumstances when the risk cannot be well determined.

Furthermore, the status of a discovered item will fall into one of the following four categories:

- **Undetermined** No response yet.
- **Acknowledged** The item has been received by the client, but not confirmed yet.
- **Confirmed** The item has been recognized by the client, but not fixed yet.
- **Fixed** The item has been confirmed and fixed by the client.

²https://owasp.org/www-community/OWASP_Risk_Rating_Methodology

³<https://cwe.mitre.org/>

Chapter 2 Findings

In total, we find **three** potential issues. Besides, we also have **six** recommendations and **three** notes.

- High Risk: 1
- Low Risk: 2
- Recommendation: 6
- Note: 3

ID	Severity	Description	Category	Status
1	High	Potential risk of asset loss due to front-running	DeFi Security	Confirmed
2	Low	Bypassable requestId check in the spotSwap() function	DeFi Security	Confirmed
3	Low	Lack of whitelist check in the depositETH() function	DeFi Security	Confirmed
4	-	Remove unused functions and events	Recommendation	Confirmed
5	-	Implement balance validation logic in the withdrawERC20() function	Recommendation	Confirmed
6	-	Add non-zero value checks for the amount parameter in some functions	Recommendation	Confirmed
7	-	Avoid using the transfer() function for native token transfers	Recommendation	Confirmed
8	-	Add a check to validate the exchangeData parameter	Recommendation	Confirmed
9	-	Add _disableInitializers() in the constructor() functions	Recommendation	Confirmed
10	-	Previously signed withdrawal transactions might become invalid after modifying important parameters	Note	-
11	-	The dailyWithdrawLimit should be set immediately after configuring the tokenWithdrawLimit	Note	-
12	-	Potential centralization risk	Note	-

The details are provided in the following sections.

2.1 DeFi Security

2.1.1 Potential risk of asset loss due to front-running

Severity High

Status Confirmed

Introduced by Version 1

Description In the `DEXVaultRouter` contract, the function `swapWithPermitToUSDTAndDeposit()` first grants token approval by invoking the `permit()` function, then conducts the token swap

by invoking the `swapToUSDT()` function. The `swapToUSDT()` function transfers tokens into the contract and performs the swap based on the `exchangeData` parameter controlled by the user. However, this process exposes a security risk. Specifically, if someone sends a transaction to call this function, an attacker could front-run the transaction (by observing pending transactions in the mempool) and submit malicious `exchangeData` along with setting `minReturnAmount` to 0. This would allow the attacker to control the execution flow of the token swap, potentially locking the user's tokens in this contract, which does not have functions to transfer or withdraw tokens. Additionally, this process is vulnerable to a sandwich attack, where an attacker places their own transactions both before and after the victim's swap transaction. By exploiting price slippage, the attacker can profit at the victim's expense.

```
89  function swapWithPermitToUSDTAndDeposit(  
90      address owner,  
91      address token,  
92      uint256 amount,  
93      uint256 minReturnAmount,  
94      uint256 deadline,  
95      uint8 v,  
96      bytes32 r,  
97      bytes32 s,  
98      bytes calldata exchangeData  
99  ) public nonReentrant {  
100      require(amount > 0, "Deposit amount must be greater than zero");  
101  
102  
103      // usdc  
104      IERC20Permit(token).permit(  
105          owner,  
106          address(this),  
107          amount,  
108          deadline,  
109          v,  
110          r,  
111          s  
112      );  
113  
114  
115      // only for permit erc20  
116      uint256 usdtAmount = swapToUSDT(  
117          owner,  
118          token,  
119          false,  
120          amount,  
121          minReturnAmount,  
122          exchangeData  
123      );  
124      IDEXVault vault = IDEXVault(VAULT_ADDRESS);  
125  
126  
127      IERC20(USDT_ADDRESS).safeIncreaseAllowance(VAULT_ADDRESS, usdtAmount);  
128
```



```
129
130     vault.depositERC20(USDT_ADDRESS, usdtAmount, owner);
131
132
133     emit SwapToUSDTAndDeposit(owner, owner, token, amount, usdtAmount);
134 }
```

Listing 2.1: DEXVaultRouter.sol

Impact Users' funds may be locked or sandwich-attacked by front-running.

Suggestion Add an `owner == msg.sender` check to mitigate the risk of front-running.

Feedback from the Project Adding `minReturnAmount` to the signature can help prevent front-running, but such attacks lack profit incentives and would only cause disruption.

2.1.2 Bypassable requestId check in the `spotSwap()` function

Severity Low

Status Confirmed

Introduced by Version 1

Description In the `DEXSpotVault` contract, the `spotSwap()` function invokes the `checkRequestId()` function to verify if the `requestId` has already been processed. However, this constraint can be bypassed because any `requestId` not in the `usedRequestIds` list will return true. Furthermore, the use of `requestId` in the function might not be necessary, as it is only used to emit the `Swap` event.

```
71     function spotSwap(
72         address owner,
73         uint256 requestId,
74         address tokenIn,
75         uint256 amountIn,
76         address tokenOut,
77         uint256 minReturnAmount,
78         bytes calldata exchangeData
79     ) public nonReentrant validSigner {
80
81
82         //check requestId
83         require(checkRequestId(requestId), "invalid request id");
84
85         require(IERC20(tokenIn).balanceOf(address(this)) >= amountIn, "vault has not enough
86             token");
87
88         uint256 amountOut = swap(
89             tokenIn,
90             amountIn,
91             tokenOut,
92             minReturnAmount,
93             exchangeData
94         );
```

```
95         emit Swap(  
96             owner,  
97             requestId,  
98             tokenIn,  
99             amountIn,  
100            tokenOut,  
101            amountOut  
102        );  
103    }
```

Listing 2.2: DEXSpotVault.sol

```
227     function checkRequestId(uint256 requestId) internal returns (bool) {  
228         if (usedRequestIds[requestId]) {  
229             return false; // Request ID has already been used  
230         }  
231         usedRequestIds[requestId] = true; // Mark this request ID as used  
232         return true;  
233     }
```

Listing 2.3: DEXSpotVault.sol

Impact The check based on the `checkRequestId()` function can be bypassed.

Suggestion Modify the logic to ensure the `requestId` is properly validated, or remove it if it is not necessary.

Feedback from the Project The `requestId` is used to prevent backend logic errors.

2.1.3 Lack of whitelist check in the `depositETH()` function

Severity Low

Status Confirmed

Introduced by Version 1

Description The `depositETH()` function in the `DEXVaultV2` contract allows anyone to deposit ETH. However, unlike the restrictions in the `depositERC20()` function, this function does not enforce the token whitelist check (`tokenWhitelist(address(0))`). As a result, users can directly deposit ETH into the `DEXVaultV2` contract after initialization, which may not align with the intended behavior of only allowing specific tokens for deposits.

```
168     function depositETH(  
169         address receiver  
170     ) public payable whenNotPaused nonReentrant {  
171         require(msg.value > 0, "Deposit amount must be greater than zero");  
172         emit Deposit(msg.sender, receiver, address(0), msg.value);  
173     }
```

Listing 2.4: DEXVaultV2.sol

Impact Users can always deposit ETH without any restrictions.

Suggestion Add a `tokenWhitelist(address(0))` check to the `depositETH()` function.

Feedback from the Project This is an extension and within expectations.

2.2 Additional Recommendation

2.2.1 Remove unused functions and events

Status Confirmed

Introduced by [Version 1](#)

Description There are some unused functions and events in the project:

- DEXSpotVault.sol
 - function `pause()`
 - function `unpause()`
- DEXVaultV2.sol
 - event `TransferToSpotVault()`

Suggestion Remove the unused functions and events for clarity.

2.2.2 Implement balance validation logic in the `withdrawERC20()` function

Status Confirmed

Introduced by [Version 1](#)

Description The `withdrawERC20()` function in the `DEXSpotVault` contract currently lacks balance validation. It is recommended to align the balance check logic with the logic used in the `withdrawERC20TokenByOwner()` function.

```
156 function withdrawERC20(  
157     address owner,  
158     address to,  
159     uint256 amount,  
160     address token  
161 )  
162     public onlyVault nonReentrant  
163  
164 {  
165     // Success, send ERC20 token  
166     IERC20(token).safeTransfer(to, amount);  
167     emit Withdraw(owner, to, token, amount );  
168 }
```

Listing 2.5: DEXSpotVault.sol

Suggestion Add balance check logic to `withdrawERC20()` function.

2.2.3 Add non-zero value checks for the amount parameter in some functions

Status Confirmed

Introduced by [Version 1](#)

Description Some functions, e.g., the `swapToUSDTAndDeposit()` function in the `DEXVaultRouter` contract, currently lack validation for the `amount` parameter. It is recommended to add a check to ensure that the deposit amount is greater than zero.

```
51 function swapToUSDTAndDeposit(  
52     address receiver,  
53     address token,  
54     uint256 amount,  
55     uint256 minReturnAmount,  
56     bytes calldata exchangeData  
57 ) public payable nonReentrant {  
58     bool isNativeToken = isNative(IERC20(token));  
59     if (isNativeToken) {  
60         require(msg.value == amount, "msg.value must be equal to amount");  
61     } else {  
62         require(msg.value == 0, "msg.value must be equal to 0");  
63     }  
64  
65     uint256 usdtAmount = swapToUSDT(  
66         msg.sender,  
67         token,  
68         isNativeToken,  
69         amount,  
70         minReturnAmount,  
71         exchangeData  
72     );  
73     IDEXVault vault = IDEXVault(VAULT_ADDRESS);  
74  
75     IERC20(USDT_ADDRESS).safeIncreaseAllowance(VAULT_ADDRESS, usdtAmount);  
76  
77     vault.depositERC20(USDT_ADDRESS, usdtAmount, receiver);  
78  
79     emit SwapToUSDTAndDeposit(  
80         msg.sender,  
81         receiver,  
82         token,  
83         amount,  
84         usdtAmount  
85     );  
86 }
```

Listing 2.6: DEXVaultRouter.sol

Suggestion Add a check to ensure that `amount` is greater than zero.

2.2.4 Avoid using the `transfer()` function for native token transfers

Status Confirmed

Introduced by [Version 1](#)

Description In the `withdrawETHByOwner()` function, the `payable(to).transfer(address(this).balance)` statement is used to send the contract's ETH balance to the destination address. However, the `transfer()` function forwards a fixed gas stipend of 2300 gas, which may not be sufficient in some circumstances, especially if the recipient is a contract that requires more gas

to process the transaction. It is advised to use the `call()` function (or the `sendValue()` function provided by OpenZeppelin) to transfer the native tokens.

```
339 function withdrawETHByOwner(  
340     address to,  
341     uint256 amount  
342 ) external onlyOwner nonReentrant returns (bool) {  
343     uint256 balance = address(this).balance;  
344     require(balance >= amount, "NOT_ENOUGH_BALANCE");  
345     payable(to).transfer(address(this).balance);  
346     return true;  
347 }
```

Listing 2.7: DEXVaultV2.sol

Suggestion Replace the `transfer()` function with the `call()` function.

2.2.5 Add a check to validate the `exchangeData` parameter

Status Confirmed

Introduced by Version 1

Description The `swap()` function in the `DEXSpotVault` contract and the `swapToUSDT()` function in the `DEXVaultRouter` contract directly pass the `exchangeData` provided by the user to 1inch's `AggregationRouter` without any validation. This lack of validation introduces the risk that incorrectly formatted `exchangeData` could cause the swapped token to be inconsistent with the specified `tokenOut` (e.g., USDT), leading to potential asset loss for users. It is recommended to implement validation logic for `exchangeData` to enforce consistency.

```
105 function swap(  
106     address tokenIn,  
107     uint256 amountIn,  
108     address tokenOut,  
109     uint256 minReturnAmount,  
110     bytes calldata exchangeData  
111 ) internal returns (uint256 returnAmount) {  
112  
113     uint256 beforeSwapBalance = IERC20(tokenOut).balanceOf(  
114         address(this)  
115     );  
116  
117     IERC20(tokenIn).safeIncreaseAllowance(  
118         AGGREGATION_ROUTER_ADDRESS,  
119         amountIn  
120     );  
121  
122     // Swap token  
123     (bool success, ) = AGGREGATION_ROUTER_ADDRESS  
124         .call(exchangeData);  
125  
126     require(success, "exchange failed");  
127 }
```

```
128     uint256 afterSwapBalance = IERC20(tokenOut).balanceOf(
129         address(this)
130     );
131
132     require(
133         afterSwapBalance > beforeSwapBalance,
134         "received token less than 0"
135     );
136
137     returnAmount = afterSwapBalance - beforeSwapBalance;
138
139     require(
140         returnAmount >= minReturnAmount,
141         "received token less than minReturnAmount"
142     );
143     return returnAmount;
144 }
```

Listing 2.8: DEXSpotVault.sol

```
131 function swapToUSDT(
132     address owner,
133     address token,
134     bool isNativeToken,
135     uint256 amount,
136     uint256 minReturnAmount,
137     bytes calldata exchangeData
138 ) internal returns (uint256 returnAmount) {
139     if (token == USDT_ADDRESS) {
140         IERC20(USDT_ADDRESS).safeTransferFrom(owner, address(this), amount);
141         return amount;
142     }
143
144     uint256 beforeSwapBalance = IERC20(USDT_ADDRESS).balanceOf(
145         address(this)
146     );
147
148     if (!isNativeToken) {
149         IERC20(token).safeTransferFrom(owner, address(this), amount);
150         IERC20(token).safeIncreaseAllowance(
151             AGGREGATION_ROUTER_ADDRESS,
152             amount
153         );
154     }
155
156     // Swap token
157     (bool success, ) = AGGREGATION_ROUTER_ADDRESS
158         .call{value: msg.value}(exchangeData);
159
160     require(success, "exchange failed");
161
162     uint256 afterSwapBalance = IERC20(USDT_ADDRESS).balanceOf(
163         address(this)
```

```
164     );
165
166     require(
167         afterSwapBalance > beforeSwapBalance,
168         "received USDT less than 0"
169     );
170
171     returnAmount = afterSwapBalance - beforeSwapBalance;
172
173     require(
174         returnAmount >= minReturnAmount,
175         "received USDT less than minReturnAmount"
176     );
177     return returnAmount;
178 }
```

Listing 2.9: DEXVaultRouter.sol

Suggestion Add validation logic to ensure consistency.

2.2.6 Add `_disableInitializers()` in the `constructor()` functions

Status Confirmed

Introduced by Version 1

Description The `DEXSpotVault`, `DEXVaultRouter`, and `DEXVaultV2` contracts all inherit the `UUPSUpgradeable` contract provided by OpenZeppelin for contract upgrades. However, currently, these contracts do not implement the `constructor()` functions. To prevent initialization of the implementation contracts, it is recommended to call the `_disableInitializers()` function inside the `constructor()` functions.

Suggestion Add `_disableInitializers()` in the `constructor()` functions.

2.3 Note

2.3.1 Previously signed withdrawal transactions might become invalid after modifying important parameters

Description In the `DEXVaultV2` contract, the contract owner can change important parameters, including updating signers by calling the `changeSigners()` function or modifying the withdrawal limit by invoking the `setWithdrawLimit()` function. This means that all previously signed withdrawal transactions might become invalid after these updates. For example, pending withdrawal requests that have already been signed by the old signers will no longer be executable.

2.3.2 The `dailyWithdrawLimit` should be set immediately after configuring the `tokenWithdrawLimit`

Description In the `DEXVaultV2` contract, when a user deposits the ERC-20 token, the `tokenWhitelist()` modifier is used to check if the ERC-20 token is in the whitelist. When a user

withdraws ERC - 20 tokens from the contract, the `dailyLimitNotExceeded()` modifier is used to check whether the token has reached the maximum withdrawal limit for the day. This creates a scenario, as seen in the `initialize()` function, where the owner sets the `tokenWithdrawLimit` for a particular token to be greater than 0, meaning the token is in the whitelist, and users can deposit this token into the contract. However, if the owner does not set the `dailyWithdrawLimit` for that token, users will not be able to withdraw that token from the contract. To avoid this issue, the owner should manually set the `dailyWithdrawLimit` immediately after configuring the `tokenWithdrawLimit`.

2.3.3 Potential centralization risk

Description In the current implementation, multiple privileged functions (e.g., `changeSigners()`, `setWithdrawLimit()` and `setDailyWithdrawLimit()`) are used to set or update critical configurations. Additionally, the owner of the `DEXVaultV2` contract can withdraw any ERC - 20 tokens or ETH through the `withdrawERC20TokenByOwner()` and `withdrawETHByOwner()` functions. If the private keys associated with these privileged roles are lost or maliciously exploited, it could potentially lead to significant losses for users.

